

形式化类群计算与 Mordell 椭圆曲线上的整数点

Anne Baanen*

Alex J. Best

Nirvana Coppola

Sander R. Dahmen

t.baanen@vu.nl

alex.j.best@gmail.com

nirvanac93@gmail.com

s.r.dahmen@vu.nl

阿姆斯特丹自由大学 (Vrije Universiteit Amsterdam)

阿姆斯特丹, 荷兰

Abstract

丢番图方程 (Diophantine equations) 是数论中一个流行且活跃的研究领域。在本文中, 我们考虑 Mordell 方程, 其形式为 $y^2 = x^3 + d$, 其中 d 是一个 (给定的) 非零整数, 且要求出所有整数 x 和 y 的解。解决这个问题的一种非初等方法是通过下降法和类群 (class groups) 进行求解。沿着这个思路, 我们在 Lean 3 中形式化了几个 $d < 0$ 实例的 Mordell 方程的求解过程。为了实现这一点, 我们需要形式化数论中其他几个本身也很有趣的理论, 例如理想范数、二次域和二次环, 以及类数的显式计算。此外, 我们引入了新的计算策略 (tactics), 以便在二次环及更一般的环中高效地进行计算。

Keywords: 形式化数学, 代数数论, 丢番图方程, 策略 (tactics), Lean, Mathlib

1 简介

对丢番图方程的研究构成了现代代数数论和算术几何的很大一部分 [6]。与相关的数学领域相比, 交互式定理证明器 (interactive theorem prover) 中涵盖的该主题内容相对较少。这部分原因可能是缺乏对超越初等数论工作所必需的形式化前置知识, 例如包括类群和单位群在内的代数数域理论。此外, 许多丢番图方程需要通过显式计算才能有效求解。如今, 这些计算通常由尚未被形式化验证的计算机代数系统执行, 如 Mathematica、Magma、Pari/GP 和 SageMath。

在本文中, 我们研究了一类被称为 Mordell 方程的丢番图方程族,

$$y^2 = x^3 + d, \quad x, y \in \mathbb{Z}, \quad (1)$$

并针对非零参数 $d \in \mathbb{Z}$ 的多个实例, 形式化了其解的完整描述 (第 5 节)。我们基于一些基本的代数数论知识 (第 2 节), 使用二次数域 (第 3 节) 和类群 (第 4 节) 的显式计算对解进行分类。这种计算以前从未在证明助手中进行过。这些论证的复杂性要求仔细建立理论并开发抽象概念, 以证明这些结果而不会产生极大的开销。

我们报告了形式化是如何进行的, 并重点介绍了我们工作的三个重要输出: 首先, 形式化了一些非平凡的丢番图方程的显式解。其次, 我们开发了其他几个具有独

立意义的理论, 例如二次域理论、理想范数理论以及类群计算。第三, 我们还建立了计算结构并添加了策略, 以在交换环的扩张中执行计算 (第 6 节), 并与计算机代数系统进行交互 (第 7 节)。只要可能, 我们的目标是在一定的通用性水平上工作, 使其适用范围超出我们考虑的 Mordell 方程所需的范围。我们形式化解决的显著困难包括: 在二次环的相同设置中发展一般理论并计算特定数值, 通过类型类 (typeclass) 系统中的计算解决附加条件 (第 3.1 和 3.2 节), 以及通过显式计算类群的方法避免使用更高级的数的几何 (geometry of numbers) (第 4.1 节)。我们希望这将为 (计算) 代数数论的进一步形式化发展, 特别是许多其他有趣的丢番图方程的显式求解铺平道路。

从 Dedekind 环的基本理论到全局域类群的有限性, 此前已由 Baanen、Dahmen、Narayanan 和 Nuccio 形式化 [3], 作为 mathlib 库 [30] 的一部分 (使用 Lean 定理证明器 [8] 的第 3 版)。我们在此基础上开展工作, 作为我们努力的一部分, 显式计算了几个二次数域的类群, 并充实了周围的理论。我们形式化的完整源代码可在线获取。¹

2 数学背景

我们用来研究丢番图方程解 (尤其是为了证明其无解) 的基本工具之一是下降法 (method of descent)。其主要思想是假设存在一个解, 并选择它在某种意义上是最小的。然后, 对这个解进行变换, 以“下降”到方程的另一个比前一个更小的解, 从而与它的最小性相矛盾。如果变换可以无条件应用, 则方程无解; 否则, 如果方程有解, 它将对解产生条件约束。

费马曾用这种方法证明了他著名的最后定理的一个实例, 即方程

$$x^4 + y^4 = z^4 \quad (2)$$

没有非零整数解。更准确地说, 费马在他的丢番图《算术》笔记中证明了 ([14, 第 1.6 节]) 不存在面积为平方数的直角三角形。这可以重述为方程 $a^4 - b^4 = c^2$ 不存在非零整数解, 进而可以推导出 (2) 没有非零解。

下降法中的关键步骤是应用以下性质: 如果 a, b, c 是三个整数且 n 是一个自然数, 使得 $\gcd(a, b) = 1$ 且 $ab = c^n$,

*作者按字母顺序排列

¹<https://github.com/lean-forward/class-group-and-mordell-equation>

那么存在 $c_1, c_2 \in \mathbb{Z}$ 使得 $a = \pm c_1^n$ 且 $b = \pm c_2^n$ 。然而，通常更方便的是不在环 $\mathbb{Z}$ 上，而是在某个更大的环（例如二次数域的整数环）上使用下降法。

我们现在回顾一些有用的定义。我们假设读者熟悉基本的环和域理论，如许多（本科）教材中所述，包括 [12, 16]。数域 (number field) K 是域 $\mathbb{Q}$ 的有限扩张。它是 $\mathbb{Q}$ 上的有限维向量空间，其维数称为 K （在 $\mathbb{Q}$ 上）的次数 (degree)。数域的例子包括 $\mathbb{Q}$ 本身（次数为 1）， $\mathbb{Q}(\sqrt{5}) = \{a + b\sqrt{5} : a, b \in \mathbb{Q}\}$ 和 $\mathbb{Q}(\sqrt{-5}) = \{a + b\sqrt{-5} : a, b \in \mathbb{Q}\}$ （次数均为 2），以及 $\mathbb{Q}(\sqrt[4]{17}) = \{a + b\sqrt[4]{17} + c\sqrt[4]{17}^2 + d\sqrt[4]{17}^3 : a, b, c, d \in \mathbb{Q}\}$ （次数为 4）。我们将主要使用二次域 (quadratic fields)，即次数为 2 的数域（第 3 节），如上面的第二个和第三个例子。一般而言，任何二次数域都同构于形式为 $\mathbb{Q}(\sqrt{d}) = \{a + b\sqrt{d} : a, b \in \mathbb{Q}\}$ 的域，其中整数 $d \neq 1$ 且无平方因子（即不被任何素数 p 的 p^2 整除）。换句话说，这些二次数域是通过添加多项式 $X^2 - d$ 的（复）根 $\alpha = \sqrt{d}$ 构造的，由于对 d 施加的条件，该多项式在 $\mathbb{Q}$ 上是不可约的。这可以如下推广到任意次数 $n \in \mathbb{Z}_{>0}$ 。对于任何在 $\mathbb{Q}$ 上不可约的 n 次多项式，将其任何一个（复）根 α 添加到 $\mathbb{Q}$ 中，将产生 n 次数域 $\mathbb{Q}(\alpha) = \{a_0 + a_1\alpha + \dots + a_{n-1}\alpha^{n-1} : a_0, a_1, \dots, a_{n-1} \in \mathbb{Q}\}$ （对 α 的 n 种不同选择都将产生同构的域）。反过来，任何 n 次数域都同构于这样一个域 $\mathbb{Q}(\alpha)$ 。

从代数和算术的角度来看，（有理）整数 $\mathbb{Z}$ 在其分式域（有理数 $\mathbb{Q}$ ）中构成了一个特别“好”的子环。在从 $\mathbb{Q}$ 推广到任意数域 K 时， $\mathbb{Z}$ 的类似物是数域 K 的整数环 (ring of integers)，记作 $\mathcal{O}_K$ 。它被定义为 $\mathbb{Z}$ 在 K 中的整闭包：

$$\mathcal{O}_K := \{x \in K : \exists f \in \mathbb{Z}[X] \text{ 首一多项式, 使得 } f(x) = 0\},$$

其中我们回顾，如果（一元）多项式的最高次项系数等于 1，则称其为首一的 (monic)。事实上， $\mathcal{O}_K$ 确实是一个环，例如可以从整闭包的一般性质（[25, 第 I.2 节]）得出。

一些整数环的例子如下。 $\mathbb{Q}$ 的整数环确实是 $\mathbb{Z}$ 。对于 $K = \mathbb{Q}(\sqrt{d})$ ，当 $d \in \{-5, -2, -1, 2, 3\}$ 时，简单地给出 $\mathcal{O}_K = \mathbb{Z}[\sqrt{d}] = \{a + b\sqrt{d} : a, b \in \mathbb{Z}\}$ ，但对于某些其他 d 值，例如 $d \in \{-3, 5\}$ ，给出 $\mathcal{O}_K = \mathbb{Z}[\frac{1}{2}(1 + \sqrt{d})] = \{a + b\frac{1}{2}(1 + \sqrt{d}) : a, b \in \mathbb{Z}\}$ 。例如， $\mathbb{Q}(\sqrt{-3})$ 的整数环严格大于 $\mathbb{Z}[\sqrt{-3}]$ ，这一事实显而易见，因为 $\frac{1}{2}(1 + \sqrt{-3})$ （它不是 $\mathbb{Z}[\sqrt{-3}]$ 的元素）是具有整数系数的首一多项式 $X^2 - X + 1$ 的根。最后，对于 $K = \mathbb{Q}(\sqrt[4]{17})$ ，我们有 $\mathcal{O}_K = \{a + b\sqrt[4]{17} + c\frac{1}{2}(\sqrt[4]{17}^2 - 1) + d\frac{1}{4}(\sqrt[4]{17}^3 + \sqrt[4]{17}^2 + \sqrt[4]{17} + 1) : a, b, c, d \in \mathbb{Z}\}$ ，它不能写成对于任何 $\alpha \in \mathcal{O}_K$ 的 $\mathbb{Z}[\alpha]$ 形式，这说明确定整数环可能很快变得复杂。

前面提到的“好”的性质，可以通过以下陈述使其精确化：数域 K 的整数环 $\mathcal{O}_K$ 是一个所谓的 Dedekind 环 (Dedekind domain) [3, 第 2 节]。虽然 $\mathcal{O}_K$ 不一定是唯一分解整环 (UFD)，但在 $\mathcal{O}_K$ （或更一般地在任何 Dedekind 环中）的非零理想，在不考虑因子顺序的情况下，可以唯一地分解为（必然非零的）素理想。

当 D 是 UFD（将 ± 1 替换为 D 中的单位）时，下降法中的关键推论在一般情况下仍然成立，例如当 $D = \mathbb{Z}[i]$

是高斯整数时，但当 D 不是 UFD（例如当 $D = \mathbb{Z}[\sqrt{-5}]$ ）时，就不再适用了。然而，当 D 是一个 Dedekind 环（例如数域的整数环）时，我们对 D 中的理想 I, J, L 有一个下降法；即以下推论对任何自然数 n 都成立：

$$\gcd(I, J) = 1 \wedge IJ = L^n \implies I = L_1^n \wedge J = L_2^n \quad (3)$$

其中 L_1, L_2 是 D 中的某些理想。

为了从理想回到元素， D 的类群 (class group)（记作 $\text{Cl}(D)$ ）至关重要。回顾 D 的分式理想 I 是 K （ D 的分式域）的 D -子模，其形式为 $I = xJ$ ，其中 $x \in K$ 且 J 是 D 的理想。令 $x = 1$ 表明每个理想 $J \subseteq D$ 也是一个分式理想；为了清晰起见，我们将这种情况称为整理理想 (integral ideal)。类群被定义为 D 的非零分式理想模掉形如 $\langle \alpha \rangle$ 的非零分式理想的等价类构成的群，以理想乘法为群运算。具体来说，如果存在非零 $\alpha \in K$ 使得 $I = \langle \alpha \rangle J$ [3, 第 2 节]，我们就说分式理想 $I \sim J$ ，其中我们用 $\langle S \rangle$ 表示由集合 S 生成的分式理想（并用 $\langle a_1, a_2, \dots, a_n \rangle$ 表示由集合 $\{a_1, a_2, \dots, a_n\}$ 生成的分式理想）。特别地， D 的非零（分式或整）理想 I 代表 D 的类群中的单位元当且仅当 I 是主理想。我们注意到 D 的类群中的每个等价类都有一个代表它的整理理想，并且可以通过取 D 的非零整理理想乘法幺半群的适当等价类 ($I \sim J \iff aI = bJ$ ，对于某些非零 $a, b \in D$) 给出类群的等价定义。

我们考虑当前的情况，其中 D 是数域的整数环，对于该数域，基本定理成立，即 D 的类群是有限的，正如 Baanen、Dahmen、Narayanan 和 Nuccio 所形式化的 [3]。 D 的类群的阶，称为 D 的类数 (class number) 并记作 $h(D) := \#\text{Cl}(D)$ ，它是衡量 D 中唯一分解失效程度的指标。特别地， $h(D) = 1$ （即类群 $\text{Cl}(D)$ 是平凡的）当且仅当 D 是一个 UFD。例如 $h(\mathbb{Z}) = h(\mathbb{Z}[i]) = 1$ 。事实证明 $\mathbb{Z}[\sqrt{-5}]$ 不是一个 UFD，因为例如 $I := \langle 1 + \sqrt{-5}, 2 \rangle$ 是一个非主理想，事实上 $\mathbb{Z}[\sqrt{-5}]$ 的任何非主（分式）理想都等价于 I 。因此 $\text{Cl}(\mathbb{Z}[\sqrt{-5}])$ 的一个完整代表系由 $\{1, I\}$ 给出，且 $h(\mathbb{Z}[\sqrt{-5}]) = 2$ 。

对于我们求解 Mordell 方程的方法所需的一个重要性质（从类群的定义和群论中的拉格朗日定理得出），对于 D 中的理想 I ：

$$\gcd(n, h(D)) = 1 \wedge I^n \text{ 是主理想} \implies I \text{ 是主理想} \quad (4)$$

最后，我们注意到数域 K 唯一确定了其整数环 D ，因此也确定了 $\text{Cl}(D)$ 和 $h(D)$ 。因此，也使用 K 的类群（记作 $\text{Cl}(K) := \text{Cl}(D)$ ）和 K 的类数（记作 $h(K) := h(D)$ ）的术语。

急切希望看到上述内容在 Mordell 方程 (1) 的显式背景下应用的读者，可以在继续下一节之前，先跳到定理 5.1 及其证明草图。

3 二次环与二次域

对 Mordell 方程进行代数求解的关键是能够方便地处理数域和数环，例如 $\mathbb{Q}(\sqrt{-7})$ 和 $\mathbb{Z}[\frac{1}{2}(1 + \sqrt{5})]$ ，这些是通过向一个环添加二次多项式的根而得到的。有几种构造方法可以使二次环的概念更加精确。将多项式 $f \in R[X]$ 的根添加到整环 R 的一种方法是取 $(R \text{ 的分式域的})$ 代数闭包（域）扩张 K ，根据定义它包含 f 的一个根 α ，并

令 $R[\alpha]$ 为 K 中包含 R 的每个元素以及 α 的最小子环。特别地，这种方法对于数环和数域效果很好，因为在这种设置下，我们总是可以将 K 设为复数域。或者，我们可以通过将 $R[\alpha]$ 设为商环 $R[X]/\langle f \rangle$ 来将根 α 添加到 R 中，其中 α 是 $X \pmod{f}$ 的等价类。这两种构造方法以前都在 `mathlib` 中可用并被使用过，在不同环境下的库中都有应用。

此外，`mathlib` 包含了用于环 $\mathbb{Z}[\sqrt{d}]$ 的结构 `zsqrtd`，它被用于定义高斯整数，并证明了一些与 Pell 方程相关的结论。但它的缺点是只能将整数的平方根添加到整数中，因此不能使用该结构形成二次域，或者环 $\mathbb{Z}[\frac{1}{2}(1 + \sqrt{5})]$ 。

上述一般方法对于我们的目的来说也有同样的两个缺点。技术上的反对意见是，新环 $R[\alpha]$ 的定义实质上利用了 R 上的环结构，包括类型 $R[\alpha]$ 中环结构的定义。由于 Lean 的合一 (unification) 算法在复杂类型上表现不佳，我们希望尽可能避免在类型中进行大型定义。更严重的反对意见是，这两种构造都不适合进行高效计算。通常，`mathlib` 在定义数学对象时很少考虑可计算性：证明中的计算是在策略执行期间通过构造证明项 (proof term) 来完成的。尽管如此，策略仍然需要一些有用的数据结构来执行其任务，并需要一种将表达式反映到此数据类型中的方法。由于复数上没有非平凡的判定程序 (decision procedures)，如果我们将 $R[\alpha]$ 定义为复数的子环，我们将无法轻易地对元素进行计算。如果我们将 $R[\alpha]$ 定义为商环 $R[X]/\langle f \rangle$ ，则可以通过选取每个等价类的代表元来进行计算。然而，使用 $R[X]/\langle f \rangle$ 进行计算将是不切实际的，因为 Lean 不知道它可以降低多项式的次数以获得更简单的代表元。此外，`mathlib` 通常不在其多项式环的实现中提供有效算法，这意味着我们无论如何都无法利用目前多项式的实现原生执行计算，而且我们将不得不在策略中重新实现多项式的算术运算。

相反，对于我们的项目，遵循一种已有的模式（类似于在 `mathlib` 中定义复数或四元数代数），我们引入了一种新的、更专业的方法将二次多项式的根添加到一个环中，我们将其命名为 `quad_ring`。

```
structure quad_ring (R : Type*) (a b : R) :=
  (b1 : R) (b2 : R)
```

上面的代码定义了一个新的类型构造器 `quad_ring`，使得 `quad_ring R a b` 的一个元素是由两个元素 `b1 b2 : R` 组成的元组，其中 R 是一个交换环（实际上，交换半环也可以）。一个元素 $\langle b1, b2 \rangle$ 表示 $b_1 + b_2\alpha \in R[\alpha]$ ，其中 α 满足代数关系 $\alpha^2 = a\alpha + b$ 。我们定义了到 `quad_ring` 的强制转换 (coercion)，允许我们隐含地将一个元素 $x : R$ 也视为 `quad_ring R a b` 的元素。

这种类型实际上表示了环 $R[\alpha]$ ，这一事实可以从环结构的定义得出，省略了一些先前的实例，它看起来如下：

```
instance : comm_ring (quad_ring R a b) :=
{ add := λ z w, ⟨z.b1 + w.b1, z.b2 + w.b2⟩,
  mul := ⟨λ z w, ⟨z.1 * w.1 + z.2 * w.2 * b,
    z.2 * w.1 + z.1 * w.2 + z.2 * w.2 * a⟩⟩,
  neg := has_neg.neg, sub := has_sub.sub,
  one := 1, zero := 0,
  nsmul := (•), npow := npow_rec,
```

```
nat_cast := coe, int_cast := coe, zsmul := (•),
.. sorry } -- proofs omitted
```

我们注意到在本文中，我们只为了阐述的目的而省略了证明；这里描述的所有内容在我们的工作中都已完全形式化（即没有 `sorry`）。

除了加法和乘法等基本环运算符外，我们还定义了与 $\mathbb{N}$ 和 $\mathbb{Z}$ 的标量乘法，以及从 $\mathbb{N}$ 和 $\mathbb{Z}$ 到任何 `quad_ring` 的强制转换。我们必须定义这些映射可能会令人惊讶，因为实际上我们可以为所有环 R 一般地定义一个唯一的环同态 $\mathbb{Z} \rightarrow R$ ，而且这还会在 R 上产生一个唯一的 $\mathbb{Z}$ -模结构。虽然我们可以证明这些构造是唯一的，但 Lean 并不会自动认为它们是定义相等 (definitionally equal) 的。`mathlib` 使用的遗忘继承 (forgetful inheritance) 模式 [1, 2] 指出，结构的继承，例如从 `comm_ring` (`quad_ring R a b`) 到 `module Z (quad_ring R a b)`，是不允许定义新数据的。相反，像标量乘法这样的操作必须从父级字面复制，在这种情况下是 `comm_ring` 中的 `zsmul` 字段。通过我们在 `comm_ring` 实例中自己指定操作，我们获得了对这些定义等式的控制。

3.1 $\mathbb{Z}[\alpha]$ 包含于 $\mathbb{Q}(\alpha)$ 中

在二次域 $\mathbb{Q}(\alpha) = \mathbb{Q}[X]/\langle f \rangle$ 内，我们有一个子环 $\mathbb{Z}[\alpha] = \mathbb{Z}[X]/\langle f \rangle$ 。环 R 的子环 A 的非正式描述是 R 的一个在环运算下封闭的子集。在这里我们将 $A = \mathbb{Z}[\alpha]$ 和 $R = \mathbb{Q}(\alpha)$ 定义为两个不同环的商，因此 A 不能以直接的方式成为 R 的子集；在 Lean 中，这些将对应于两种不同的类型，而不是类型 R 和项 $A : \text{set } R$ 。以类似的方式，整数 $\mathbb{Z}$ 非正式地是有理数 $\mathbb{Q}$ 的子环，但形式上不是子集。`mathlib` 相反倾向于将子环关系转化为一个单射映射：参数 $(f : A \rightarrow^+ R)$ ($hf : \text{injective } f$)，其中 f 是一个环同态，而 hf 携带一个单射性证明 [3]。在环运算下封闭的子集 A 通过使用遗忘 $x \in A$ 的强制转换映射 (ι) 来符合这种模式。

此外，给定环 A, R ，通常对这种包含 $A \rightarrow R$ 有一个典型的选择。假设 A 有一个到 R 的典型映射，由参数 `[algebra A R]` 表示，其中 `algebra (A R : Type*) [comm_semiring A] [semiring R] : Type*` 是一个依赖于两种类型 A, R 的多参数类型类。这提供了一个典型的同态 `algebra_map A R : A →+* R`；因此一个子环可以由一对参数 `[algebra A R] (h : injective (algebra_map A R))` 来表示。

在这里，方括号表示一个实例参数 (instance parameter)，这是一种隐式参数 [2]。Lean 的推导器 (elaborator) 通过依次考虑标记为 `instance` 的声明，自动为这些参数合成一个值，将它的类型与参数类型进行合一，并使用第一个匹配的实例。实例本身也可以有实例参数，其合成方式相同。在 Lean 3 中，合成被实现为深度优先搜索。这种搜索允许我们在合成期间执行 Prolog 风格的非平凡计算，正如我们将在下面讨论的那样。

在二次环的情况下，一个 `algebra` 实例将很自然地具有类型 `algebra (quad_ring Z a b) (quad_ring Q a b)`。然而，这包含了类型错误，因为表达式 `quad_ring Z a b` 中的 `a b : Z` 以及表达式 `quad_ring Q a b` 中的

$a\ b : \mathbb{Q}$ 。因此我们的第一次尝试是通过将类型为 $\mathbb{Z} \rightarrow \mathbb{Q}$ 的典型包含应用于 $a\ b$ 来定义实例，或者更一般地对于任何 A -代数 R ：

```
instance algebra1 [algebra A R] (a b : A) :
  algebra (quad_ring A a b) (quad_ring R
    (algebra_map A R a)
    (algebra_map A R b))
```

表达式 `quad_ring R (algebra_map A R a) (algebra_map A R b)` 的冗长是不令人满意的，而当系数是数字表达式时，情况会变得更糟： $\mathbb{Z}[\sqrt{5}]$ 在 $\mathbb{Q}[\sqrt{5}]$ 中的包含被记为 `algebra_map (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  0)) (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  5))`。更糟糕的是，对于 `quad_ring  $\mathbb{Q}$  0 d` 的任何结果，都需要穿过等式 `algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  0 = 0` 进行传输，然后才能应用于 `quad_ring  $\mathbb{Q}$  (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  0) (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  5))`。

为了开发一个没有这些缺陷的实例，我们将系数 a' $b' : R$ 作为单独的参数添加到 `algebra` 实例中，连同一对假设断言 a' b' 分别是 $a\ b$ 的像，以确保映射仍然是良定义的：

```
def algebra2 [algebra A R] (a b : A) (a' b' : R)
  (ha : algebra_map A R a = a')
  (hb : algebra_map A R b = b') :
  algebra (quad_ring A a b) (quad_ring R a' b')
```

对于 $a\ b$ 的数值，由 `simp` 策略调用的 Lean 简化过程足以证明等式假设，这意味着我们可以定义如下实例：

```
instance algebra_sqrt_5 :
  algebra (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  0 5) :=
  algebra2  $\mathbb{Z}$   $\mathbb{Q}$  0 5 0 5 (by simp) (by simp)
```

然而，我们无法将 `algebra2` 全局声明为一个 `instance`，因为类型类系统无法自动为 `ha hb` 参数提供值。

一般来说，为了能够推断出一个实例，实例的所有参数必须要么出现在目标的类型中，要么它们本身就是实例参数。为了将等式假设 `ha hb` 提升为实例参数，我们将其包装在 `fact` 类中 [2]。这是 `mathlib` 中使用的一个小型包装类，它允许我们将某些证明注册为实例参数：

```
class fact (p : Prop) : Prop := (out [] : p)
```

使用这个，我们的 `algebra` 实例变成了（回想一下实例参数是在 [方括号] 中传递的）

```
instance algebra3 [algebra A R]
  (a b : A) (a' b' : R)
  [fact (algebra_map A R a = a')]
  [fact (algebra_map A R b = b')] :
  algebra (quad_ring A a b) (quad_ring R a' b')
```

Lean 富有表现力的类型类系统与类似于 Prolog 的合成使其能够执行计算。这意味着我们可以定义一组实例，通过求值自动合成相等证明的 `fact` 实例，或者从另一个角度来看，我们为形式为 `algebra_map A R a = a'` 的表达式提供了一个大步操作语义 (big-step operational semantics)。立即终止的计算步骤对应于没有进一步实例参数的实例：

```
instance fact_eq_refl (a : R) : fact (a = a)
instance fact_map_zero (f : A  $\rightarrow$ +* R) :
  fact (f 0 = 0)
instance fact_map_one (f : A  $\rightarrow$ +* R) :
  fact (f 1 = 1)
```

不终止的计算步骤采用一个表示仍需完成的计算的实例参数，例如下面的 `h` 参数：

```
instance fact_map_neg (f : A  $\rightarrow$ +* R)
  (a : A) (a' : R) [h : fact (f a = a')] :
  fact (f (-a) = -a')
```

最后，为了处理数字参数，我们添加了两个实例，用于评估这些数字的二进制表示，由 `bit0` 和 `bit1` 函数给出：

```
instance fact_map_bit0 (f : A  $\rightarrow$ +* R)
  (a : A) (a' : R) [fact (f a = a')] :
  fact (f (bit0 a) = bit0 a')
instance fact_map_bit1 (f : A  $\rightarrow$ +* R)
  (a : A) (a' : R) [fact (f a = a')] :
  fact (f (bit1 a) = bit1 a')
```

`algebra3` 实例和 `fact` 集合的组合足以自动合成所需的实例：

```
instance algebra_sqrt_5' :
  algebra (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  0 5) :=
  by apply_instance
```

尽管 Lean 3 的实例合成算法缺乏 Prolog 实现中或确实在 Lean 4 合成算法中 [28] 可用的许多改进，但我们发现它足以在合理的时间内执行这组有限的计算：由于缓存，影响仅限于对包含形式为 `algebra (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  0 5)` 目标的每个声明产生大约 100 毫秒的一次性成本。对于较大的数字，成本也保持在较低水平，因为合成复杂度与二进制表示的大小呈线性关系。在我们的项目中使用这种设计，我们不需要调整实例或进行其他优化。

3.2 整数环与 Dedekind 环

建立了一般理论后，我们特化到形式为 $R[\sqrt{d}]$ 的环，特别是 $\mathbb{Z}[\sqrt{d}]$ 和 $\mathbb{Q}(\sqrt{d}) = \mathbb{Q}[\sqrt{d}]$ （在代码示例中我们也将使用符号 $\mathbb{Z}[\sqrt{d}]$ 和 $\mathbb{Q}(\sqrt{d})$ 来表示这些特定的环）。对于域 K ，我们证明如果 d 不是一个完全平方数，则 $K[\sqrt{d}]$ 是一个域（否则，该环将包含零因子）。我们首先将域结构作为一个定义提供，而不是作为一个实例，因为类型类系统并未设置为能推断给定的 d 不是一个完全平方数。

```
def field_of_not_square {K : Type*} [field K]
  {d : K} (hd :  $\neg$  is_square d) :
  field (quad_ring K 0 d)
```

因为在每次需要时手动提供域结构会很不方便，我们使用 `fact` 构造来使命题可用于类型类系统，所以我们可以定义一个 `field` 实例：

```
instance {K : Type*} [field K]
  {d : K} [hd : fact ( $\neg$  is_square d)] :
  field (quad_ring K 0 d) :=
  field_of_not_square (fact.out hd)
```

为了进一步证明 $\mathbb{Q}(\sqrt{d})$ 是 $\mathbb{Z}[\sqrt{d}]$ 的分式域, 我们再次必须小心地区分数字 $d : \mathbb{Z}$ 及其像 $d' : \mathbb{Q}$, 我们通过使用两个独立的变量并添加一个假设 `[hdd : fact (algebra_map ℤ ℚ d = d')]` 来实现这一点。我们通过注册一个类型类实例 `is_fraction_ring R K` 来表示 K 是 R 的分式域, 这意味着 K 是包含 R 的最小域; 这个事实的标准证明很容易形式化。

最后, 我们假设 d 是无平方因子的, 并证明当 $d \equiv 2$ 或 $d \equiv 3 \pmod{4}$ 时 $\mathbb{Z}[\sqrt{d}]$ 是一个 Dedekind 环, 且当 $d \equiv 1 \pmod{4}$ 时 $\mathbb{Z}[\alpha] = \mathbb{Z}[\frac{1}{2}(1 + \sqrt{d})]$ 是一个 Dedekind 环, 方法是证明在给定条件下它们是 $\mathbb{Q}(\sqrt{d})$ 的整数环。正如条件所暗示的, 这个事实的证明涉及在环 $\mathbb{Z}/4\mathbb{Z}$ 中工作。这些条件提供了一个很好的例子来说明处理强制转换时涉及的微妙之处: 例如我们可以将条件写为 $d \% 4 = 3$, 停留在 $\mathbb{Z}$ 中并允许直接计算以验证该条件, 或者我们可以写成 $(\uparrow d : \mathbb{Z} \bmod 4) = 3$, 将 d 转换为 $\mathbb{Z}/4\mathbb{Z}$ 的元素, 这在 $\mathbb{Z} \bmod 4$ 中工作时可直接适用。我们选择将这个条件表述为 $d \% 4 = 3$, 因为现有的证明自动化工具可以更容易地检查这个条件; 每当在证明中 d 被强制转换为 $\mathbb{Z}/4\mathbb{Z}$ 时, 我们可以使用 `norm_cast` 策略 [19] 来证明 $3 : \mathbb{Z}$ 可以被强制转换为 $3 : \mathbb{Z} \bmod 4$ 。

对于 $d \equiv 1 \pmod{4}$ 的情况, 我们选择 $m : \mathbb{Z}$ 使得 $d = 4m+1$, 并将 $\mathbb{Z}[\alpha]$ 写为 `quad_ring 1 m`。我们再次提供一个 `is_fraction_ring` 实例, 这次在假设 `[hdm : fact (d' = 4 * ↑m + 1)]` 下进行。虽然可以通过用 `algebra_map R K` 替换强制转换 $\mathbb{Z} \rightarrow \mathbb{Q}$ 来推广这个实例, 但我们不认为这种通用性值得用处理强制转换的简易性来换取。

所讨论的环是 Dedekind 环这一事实, 是它们作为 $\mathbb{Q}(\sqrt{d})$ 整数环的推论。换句话说, 我们必须证明一个元素 $x \in \mathbb{Q}(\sqrt{d})$ 是 $\mathbb{Z}[\sqrt{d}]$ 或 $\mathbb{Z}[\alpha]$ 的元素当且仅当 x 是整的 (integral): 即它是某个系数为整数的首一多项式的根。由于 $\sqrt{d}$ 、 α 和每个整数都满足这个性质, 并且整性在加法和乘法下是保持的, 我们可以很容易地证明 $\mathbb{Z}[\sqrt{d}]$ 或 $\mathbb{Z}[\alpha]$ 的每个元素都是整的。我们通过显式计算 $a+b\sqrt{d}$ 的极小多项式 (系数在 $\mathbb{Q}$ 中) 为 $X^2 - 2aX + a^2 - db^2$ (如果 $b \neq 0$) 来证明其逆命题; 如果 $a+b\sqrt{d}$ 是整的, 则这些系数是整数, 因为 $\mathbb{Z}$ 是具有分式域 $\mathbb{Q}$ 的欧几里得整环。如果 d 模 4 不是完全平方数 (即 $d \in \{2, 3\} \pmod{4}$), 这意味着 b 是整数, 由此得出 a 也是整数, 给出 $a+b\sqrt{d} \in \mathbb{Z}[\sqrt{d}]$ 。如果 $d \equiv 1 \pmod{4}$, 我们可以证明存在整数 x, y 使得 $a = x + \frac{1}{2}y$ 且 $b = \frac{1}{2}y$, 所以 $a+b\sqrt{d} = x + y\frac{1}{2}(1 + \sqrt{d}) \in \mathbb{Z}[\alpha]$ 。结果是这样一对定理:

```
theorem is_integral_closure_1 {m : ℤ} {d' : ℚ}
  [hdm : fact (d' = 4 * ↑m + 1)] :
  ¬is_square d' → squarefree (4 * m + 1) →
  is_integral_closure (quad_ring ℤ 1 m) ℤ
  ℚ(√d')
theorem is_integral_closure_23 {d : ℤ} {d' : ℚ}
  [hdd' : fact (algebra_map ℤ ℚ d = d')] :
  d % 4 = 2 ∨ d % 4 = 3 → squarefree d →
  is_integral_closure ℤ[√d] ℤ ℚ(√d')
```

给出了 $\mathbb{Q}(\sqrt{d})$ 的整数环的模型。

4 类群计算

由 Baanen、Dahmen、Narayanan 和 Nuccio 形式化的一个经典结果 [3], 是数域类群的有限性。我们建立在此形式化的基础上, 计算了一些非平凡的类群。虽然现有的形式化可以通过提供整数环的显式描述来变得有效, 但它提供的界限非常弱, 会导致不必要的工作, 所以我们选择明确地重做这些计算。一般来说, 得到一个带有整数环 $\mathcal{O}_K$ 的数域 K 的类群有两个基本要素。首先, 需要得到类群的一个有限生成集。一个众所周知的能得到这一结果的经典定理是 Minkowski 界, 我们将在本节末尾简要讨论。然而, 我们采用了更直接的带余除法的推广方法, 如下一小节所述。最后, 给定一个有限生成集, 我们需要确定其乘法结构以完全确定该群。在我们当前的方法中, 我们专注于包括非平凡例子在内的小例子。

4.1 上界

我们将建立在 [3, 第 8.2 节] 中关于数域 K 的类群有限性的形式化基础上。我们的起点是参考文献中形式化的定理的以下推广。

Theorem 4.1. 令 I 为 $\mathcal{O}_K$ 的一个非零理想, 令 M 为一个非零整数的有限集, 并假设对于所有 $a, b \in \mathcal{O}_K$ 且 $b \neq 0$, 存在 $q \in \mathcal{O}_K$ 和 $r \in M$ 使得

$$|\text{Norm}_{\mathcal{O}_K/\mathbb{Z}}(ra - qb)| < |\text{Norm}_{\mathcal{O}_K/\mathbb{Z}}(b)|. \quad (5)$$

那么存在 $\mathcal{O}_K$ 的一个理想 J 使得

$$I \sim J \text{ 且 } J \left| \left\langle \prod_{m \in M} m \right\rangle \right|.$$

我们注意到, 我们可以不取由 M 中所有元素的乘积生成的理想, 而是取由 M 中元素的任何非零公倍数 (例如它们的最小公倍数) 生成的理想。但是, 当我们专注于生成元时, 我们可以使用理想的唯一分解将 J 限制为素理想, 因此通过简单地对 M 中的所有元素取乘积, 我们不会引入任何更多的素理想。我们在 Lean 中形式化了上述陈述, 将 $\mathbb{Z}$ 推广到具有绝对值操作 `abv` 的任何欧几里得整环 R 并将 $\mathcal{O}_K$ 推广到任何 Dedekind 环 S , 使得 $L = \text{Frac}(S)$ 是 $K = \text{Frac}(R)$ 的有限扩张。

```
theorem exists_mk0_eq_mk0' (I : (ideal S)⁰)
  (M : finset R) :
  (∀ m ∈ M, algebra_map R S m ≠ 0) →
  (∀ (a : S) (b ≠ 0) → (∃ (q : S) (r ∈ M),
    abv (norm R (r * a - q * b)) <
    abv (norm R b))) →
  ∃ (J : (ideal S)⁰), mk0 L I = mk0 L J ∧
  algebra_map _ _ (∏ m in M, m) ∈ J
```

这里 `mk0 L` 是将一个非零理想 $I : (\text{ideal } S)^0$ 映射到其在类群中等价类的同态。

为了建立定理 4.1 中给出的广义带余除法的假设, 将整数环中的范数估计 (5) 转化为分式域中的范数估计, 两边除以因子 b 是很实用的。

```
lemma exists_lt_norm_iff_exists_le_one
  [is_integral_closure S R L]
```

```
(M : finset R) :
(∀ (a : S) (b ≠ 0), ∃ (q : S) (r ∈ M),
  abv (norm R (r · a - q · b)) <
  abv (norm R b)) ↔
(∀ (y : L), ∃ (q : S) (r ∈ M),
  abv (norm K (r · y - q · 1)) < 1)
```

在证明中，我们花了最多的精力繁琐地证明 b 非零当且仅当它在 L 中的像非零，当且仅当它在 R 上的范数非零，当且仅当它在 K 上的范数非零，以证明在两边除以 b 这一想法的合理性。

我们现在转到上述结果的具体推论。我们考虑二次数域 $K = \mathbb{Q}(\sqrt{d})$ ，其中 $d \in \mathbb{Z}$ 并假设 $d \equiv 2$ 或 $3 \pmod{4}$ 并且 d 无平方因子（除非另有明确指示），这在 Lean 中使用类型类假设 `[fact (d % 4 = 2 ∨ d % 4 = 3)]` 和 `[fact (squarefree d)]` 来指定，因为 Dedekind 环实例无论如何都需要这些假设。为了得到 $d \in \{-1, -2, -5, -6\}$ 的类群结果，我们针对 $\gamma \in \mathbb{Q}(\sqrt{d})$ 执行了必要的范数估计，并将集合 M 取为 $\{1, 2\}$ ：

```
theorem sqrt_neg.exists_q_r
[fact (algebra_map ℤ ℚ d = d')]
(γ : ℚ(√d)) : -6 ≤ d → d ≤ 0 →
∃ (q : ℤ[√d]) (r ∈ {1, 2}),
  abs (norm ℚ (r · γ - q · 1)) < 1
```

证明是初等的，基本上由根据 γ 的分量（有理数）进行的情况区分，以及为这些分量建立不等式组成。在我们建立了不等式非线性部分的估计之后，策略 `linarith` 就能解决这些目标。

将上述定理与前面提到的两个结果结合起来现在表明，对于 $d \in \{-1, -2, -5, -6\}$ ，在 $\mathbb{Z}[\sqrt{d}]$ 中的任何非零理想 I 在类群中等价于整除 $\langle 1 \rangle$ 或 $\langle 2 \rangle$ 的一个理想 J 。下一个任务是将 $\langle 2 \rangle$ 分解为素理想。为此，我们显式定义了一个非零理想 `sqrt_2 d`，使得 $(\text{sqrt_2 } d)^2$ 是由 2 生成的理想，并证明这个理想是不可约的。`sqrt_2 d` 的定义取决于 d 模 4 的余数。

```
def sqrt_2 [fact (d % 4 = 2 ∨ d % 4 = 3)] :
  (ideal ℤ[√d])0 :=
if d % 4 = 2 then
  ⟨ideal.span {(0, 1) : ℤ[√d]}, 2⟩,
  sorry -- Proof of nonzeroness omitted
else
  ⟨ideal.span {(1, 1) : ℤ[√d]}, 2⟩,
  sorry -- Proof of nonzeroness omitted
```

我们通过显式证明每个理想的生成元集合包含在另一个理想中，检查了在两种情况下 `span {2}` 都可以分解为 $(\text{sqrt_2 } d)^2$ 。检查这个等式比基于非正式论证的简单性所表现出来的要费力得多，并且每个等式的形式化都需要超过 20 行代码。这是因为有许多例行的整除性和显式的数值条件需要检查，而这些目前必须手动引导。在证明助手执行类似计算时，这是一个需要开发改进的策略或更好的抽象以减少工作量的领域。

为了证明 `sqrt_2 d` 是一个素理想，我们考虑它的范数。在代数数论中，整理理想的范数可以取为整数，即绝对理想范数 (*absolute ideal norm*)，或作为子域的理想，给出

相对理想范数 (*relative ideal norm*)。我们在 Dedekind 环 R 中将绝对理想范数形式化为一个乘法同态，对于 R 的理想 I ，如果商 R/I 是有限的，则定义为 $N(I) = |R/I|$ ，否则为 0。我们分两步证明其乘法性：首先证明当 I 和 J 互素时 $N(IJ) = N(I)N(J)$ ，然后证明当 P 是素理想且 i 是自然数时 $N(P^i) = N(P)^i$ ；Dedekind 环中理想的唯一分解性质使我们得出所有 I 和 J 都满足 $N(IJ) = N(I)N(J)$ 。

我们计算出 `sqrt_2 d` 的范数是素数 2，并得出 `sqrt_2 d` 本身就是素理想的结论：如果它分解为 IJ ，则 $N(IJ) = N(I)N(J) = 2$ ，不失一般性地意味着 $N(I) = |R/I| = 1$ ，因此 $I = R$ 。

我们得出结论，对于 $d \in \{-1, -2, -5, -6\}$ ， $\mathbb{Z}[\sqrt{d}]$ 的类群由包含 `sqrt_2 d` 的类生成，我们将其称为 `class_group.sqrt_2 d`。这个理想的平方是 2 这一事实表明这个类的阶为 1 或 2。特别地，类群最多由 2 个元素组成：

```
theorem class_group_eq (d : ℤ)
[fact (d % 4 = 2 ∨ d % 4 = 3)]
[fact (squarefree d)]
(I : class_group ℤ[√d]) :
-6 ≤ d → d ≤ 0 → I ∈ {1, sqrt_2 d}
```

类似地，但花了更多的精力，我们证明了 $\mathbb{Z}[\sqrt{-13}]$ 的类群由这两个类生成。我们取集合 $M = \{1, 2, 3, 4\}$ 进行范数估计（ $\{1, 2, 3\}$ 也可以），并利用 Kummer-Dedekind 定理证明 $\langle 3 \rangle$ 是 $\mathbb{Z}[\sqrt{-13}]$ 中的一个素理想。

对于解决本定理涵盖的那些 d 的 Mordell 方程的应用来说，这实际上是足够完成证明的信息，因为对于这些应用，唯一的要求是 3 不整除类数。然而，在一般情况下，有必要完全计算类数，而不仅仅是上限 2，因此我们还形式化了展示类群不同元素的方法，并给出了关于其大小的非平凡下界。

4.2 下界

为了完成我们对类群的计算，我们需要确定我们找到的类是否是不同的。为此，我们需要确定某些理想是否为主理想。虽然我们可以通过给出一个明确的生成元来轻松地证明它是主理想，但证明非主理想可能会比较棘手。然而，对于我们的例子，至少当 $d < -2$ 时，我们可以在相当普遍的情况下快速得到非主理想的必要证明。连同已经确立的 $(\text{sqrt_2 } d)^2$ 的主理想性质，我们得到：

```
lemma order_of_sqrt2 (d : ℤ)
[hd : fact (d % 4 = 2 ∨ d % 4 = 3)] [fact
  (squarefree d)] :
d < -2 → order_of (class_group.sqrt_2 d) = 2
```

为了证明 `sqrt_2 d` 不是主理想，我们取一个假设生成元 $a + b\sqrt{d}$ 的范数，得到一个方程 $a^2 - db^2 = 2$ ，其中 $a, b \in \mathbb{Z}$ 。一个简单的估计表明，对于 $d < -2$ ，不存在任何解。拉格朗日定理于是得出：

```
theorem two_dvd_class_number (d : ℤ) (d' : ℚ)
[fact (algebra_map ℤ ℚ d = d')] [fact (d % 4 =
  2 ∨ d % 4 = 3)] [fact (squarefree d)] :
d < -2 → 2 ∣ class_number ℚ(√d')
```


这说明对于任何无平方因子的整数 $d < -2$ 且 $d \equiv 2$ 或 $3 \pmod{4}$, 都有 $2 \mid h(\mathbb{Q}(\sqrt{d}))$ 。

最初 `mathlib` 中类群的定义取决于分式域的选择, 这意味着在形式化的最初版本中, 我们必须在 $d : \mathbb{Z}$ 和 $d' : \mathbb{Q}$ 之间保持区别。这使得我们不得不在 d 和 d' 之间反复传递假设。由于每种分式域的选择都给出了同构的类群, 我们重新定义了 `class_group`, 转而使用一个特定的构造 `fraction_ring` 作为分式域, 并在需要其他选择时插入显式的同构。这意味着我们只需在计算类数的最后一步才需要转移到 $d' : \mathbb{Q}$ 。

4.3 显式结果

对于 $d \in \{-5, -6, -13\}$, 上界和下界一起表明 $h(\mathbb{Q}(\sqrt{d})) = 2$:

```
lemma class_number_5 : class_number  $\mathbb{Q}(\sqrt{-5}) = 2$ 
lemma class_number_6 : class_number  $\mathbb{Q}(\sqrt{-6}) = 2$ 
lemma class_number_13 : class_number  $\mathbb{Q}(\sqrt{-13}) = 2$ 
```

事实上, 在这些情况下, 我们也得到了类群 $\text{Cl}(\mathbb{Q}(\sqrt{d}))$ 元素的显式 (且不同的) 代表。对于 $d \in \{-1, -2\}$, 不难证明 $\text{sqrt}_2 d$ 是主理想, 从而得出 $h(\mathbb{Q}(\sqrt{d})) = 1$:

```
lemma class_number_1 : class_number  $\mathbb{Q}(\sqrt{-1}) = 1$ 
lemma class_number_2 : class_number  $\mathbb{Q}(\sqrt{-2}) = 1$ 
```

或者, 人们可以通过令 $M = \{1\}$ 执行范数估计来更直接地获得这些 UFD 结果。

4.4 其他可能的方法

众所周知, 通过所谓的 Minkowski 界, $\mathbb{Q}$ 上 n 次数域 K 的类群由满足以下条件的素理想 P 的类生成:

$$\text{Norm}(P) \leq \sqrt{|\Delta_K|} \left(\frac{4}{\pi} \right)^{r_2} \frac{n!}{n^n} \quad (6)$$

其中 r_2 表示 $K \hookrightarrow \mathbb{C}$ 的非实复嵌入的共轭对的数量, Δ_K 表示 K 的判别式 (参见例如 [25, 第 I.5 节])。在我们的情况下, 对于 $K = \mathbb{Q}(\sqrt{d})$, 其中 $1 \neq d \in \mathbb{Z}$ 无平方因子, 我们有以下等式

$$\Delta_K = \begin{cases} d & \text{如果 } d \equiv 1 \pmod{4}, \\ 4d & \text{如果 } d \equiv 2, 3 \pmod{4}. \end{cases}$$

证明 (6) 所基于的主要结论是 Minkowski 的凸体定理, 它之前已经在简单类型论和依赖类型论中被形式化过 (例如在 Isabelle/HOL [13] 和 Lean² 中)。

虽然本节概述的方法和 Minkowski 界都适用于任何次数的数域, 但在二次域的环境下还有另一种计算类群 (或至少是类数) 的方法。在 $K = \mathbb{Q}(\sqrt{d})$ 的情况下, 对于负的无平方因子整数 d , 解析类数公式 (Analytic Class Number Formula) [25, 第 VII.5 节] 给出了一个类数的公式, 作为一个显式的有限和

$$h(K) = \frac{w}{2\Delta_K} \sum_{a=1}^{|\Delta_K|-1} \left(\frac{\Delta_K}{a} \right)_a,$$

其中 w 是 K 中单位的数量, 即 2, 除非 $d = -1$ 或 -3 (在这些情况下 w 分别为 4, 6), 而 $\left(\frac{\Delta_K}{a} \right)$ 表示雅可比符号。

将这个结果为我们的目的进行形式化的主要困难在于所需的解析结果与本项目其余部分使用的偏代数的结果相距甚远。

5 Mordell 方程的显式解

本文的重点是通过计算类群和 (理想) 下降法, 对某些 Mordell 方程 (1) 的解进行形式化。但在开始之前, 我们先简要介绍一下 Mordell 方程本身的历史。

1657 年, Fermat 在一封信中声称, 方程 $y^2 = x^3 - 2$ 在正整数中的唯一解是 $(x, y) = (3, 5)$ 。1914 年, Mordell [21] 从两个角度研究了一般方程 $y^2 = x^3 + d$: 一是使方程不存在任何整数解的关于 d 的条件; 二是基于理想下降法为某些 d 寻找解的方法。这个方程很快就被称为 Mordell 方程。1918 年, 他 [22] 证明了一个非有效 (ineffective) 的有限性定理: 对于每个非零的 $d \in \mathbb{Z}$, Mordell 方程只有有限多个整数解。另请参阅 Mordell 的著作 [24, 第 26 章]。1967 年, Baker [4] 证明了以下有效 (effective) 的结果: 对于每个 $d \in \mathbb{Z}$ ($d \neq 0$), 如果 $x, y \in \mathbb{Z}$ 满足 $y^2 = x^3 + d$, 那么

$$\max\{|x|, |y|\} \leq e^{10^{10}|d|^{10000}}. \quad (7)$$

在此基础上, 人们获得了渐近更尖锐的有效界限。最值得注意的是, Stark [29] 在 1973 年证明了: 对于任何 $\epsilon > 0$, 存在一个可有效计算的常数 C_ϵ , 使得 $\max\{|x|, |y|\} < C_\epsilon^{|d|^{1+\epsilon}}$ 。直接使用这种界限来求解任何给定 d 的 Mordell 方程是不切实际的, 但是人们已经开发出了有效解决 Mordell 方程的技术, 例如 Bennett 和 Ghadermanzi [5] 在 2015 年对所有 (非零) $|d| < 10^7$ 求解了 (1)。

对上述任何有效界限, 或提到的对 $|d| < 10^7$ 的求解过程进行形式化, 都需要首先发展大量的背景理论。作为不使用初等数论求解丢番图方程方向的一步, 我们形式化了 Mordell [21, 第 8 节] 中出现的一个灵活的利用类群下降法存在解的结论。

Theorem 5.1. 令 $d < 0$ 为一个无平方因子的整数, 且 $d \equiv 2$ 或 $3 \pmod{4}$, 并假设 $3 \nmid h(\mathbb{Z}[\sqrt{d}])$ 。那么 Mordell 方程 (1) 有解当且仅当对于某个 $m \in \mathbb{N}$ 有 $d = -3m^2 \pm 1$, 在这种情况下 $y = \pm m(3d + m^2)$ 。

现在让我们快速勾勒一下这个结论的证明:

Proof. 假设我们有 Mordell 方程 (1) 的一个解 (x, y) 。然后将 d 移到另一侧, 并在环 $D := \mathbb{Z}[\sqrt{d}]$ 上分解该多项式, 我们得到

$$(y + \sqrt{d})(y - \sqrt{d}) = x^3.$$

因此, 对于分别由 $y + \sqrt{d}$ 、 $y - \sqrt{d}$ 和 x 生成的理想 I 、 J 和 L , 我们有 $IJ = L^3$ 。在上述设置中, 理想 I 和 J 是互素的, 因为它们的最大公约数 (GCD) 同时整除 $2y$ 和 $2\sqrt{d}$; 然而, 可以证明 y 和 d 是互素的且 x 是奇数, 所以这些理想的 GCD 必须为一。因为 D 是数域 $\mathbb{Q}(\sqrt{d})$ 的整数环, 我们可以调用 (3), 它特别暗示了对于 D 中的某个理想 L_1 有 $I = L_1^3$ 。请注意, 通过构造, I 是主理想, 因此结合

²<https://github.com/leanprover-community/mathlib/pull/2819>

假设 $3 \nmid h(\mathbb{Z}[\sqrt{d}])$ ，我们从 (4) 得到 L_1 是主理想。这可以转化为对于某个 $\alpha \in D$ 和单位 $u \in D^*$ ，有 $y + \sqrt{d} = u\alpha^3$ 。由于 D 中的所有单位都是立方数，我们可以不失一般性地假设 $u = 1$ 。将 α 写为 $a + b\sqrt{d}$ ，我们得出

$$y + \sqrt{d} = (a + b\sqrt{d})^3 = a(a^2 + 3b^2d) + b(3a^2 + b^2d)\sqrt{d}.$$

比较等式左右两侧 $\sqrt{d}$ 的系数，得出

$$1 = b(3a^2 + b^2d).$$

因为这构成了 1 在 (有理) 整数 $\mathbb{Z}$ 中的分解，我们得到 (代入 $b^2 = 1$):

$$b = 3a^2 + d = \pm 1.$$

相应的 y 值现在可以被计算出来，结果也很快随之得出。 $\square$

5.1 形式化 Mordell 方程的解

我们在 Lean 中遵循上述草图，形式化了定理 5.1 的证明，作为以下陈述：

```
theorem Mordell_d (d : ℤ) (d' : ℚ) (hd : d ≤ -1)
  (hmod : d % 4 = 2 ∨ d % 4 = 3)
  (hdd' : algebra_map ℤ ℚ d = d')
  (hdsq : squarefree d) [fact (¬ is_square d')]
  (hcl : (3 : ℕ).gcd (class_number ℚ(√d')) = 1)
  (x y : ℤ) (h_eqn : y ^ 2 = x ^ 3 + d) :
  ∃ m : ℤ, ↑y = m * (d * 3 + m ^ 2) ∧
    (d = 1 - 3 * m ^ 2 ∨ d = -1 - 3 * m ^ 2)
```

请注意，正如在第 3 节中那样，我们使用 `fact` 向类型类系统呈现一个假设。

证明大致遵循刚才概述的非正式草图。首先我们分解项 $y^2 - d = (y + \sqrt{d})(y - \sqrt{d})$ ，使用算术策略 `ring` 和 `calc_tac` 来检查分解结果。然后我们过渡到在 `class_group ℤ[√d] ℚ(√d')` 中的一个等式，并应用理想下降引理 (3) 得到理想 $\langle y + \sqrt{d} \rangle$ 是一个立方。要检查理想 $\langle y + \sqrt{d} \rangle, \langle y - \sqrt{d} \rangle$ 是互素的这个附加条件，需要许多行代码，而且似乎比这些证明的非正式表达要复杂得多。这是更好的策略或抽象概念可以派上用场的情况。需要检查的陈述如下：

```
lemma gcd_lemma {x y d : ℤ} [fact (d % 4 = 2 ∨ d % 4 = 3)] [fact (squarefree d)]
  (h_eqn : y ^ 2 - d = x ^ 3) :
  gcd (span {⟨y, 1⟩}) (span {⟨y, -1⟩}) = 1
```

在纸笔证明中，作者通常取逆否命题，并假设存在一个素理想 $\mathfrak{p}$ 同时整除 $\langle y + \sqrt{d} \rangle, \langle y - \sqrt{d} \rangle$ ，并推导出与 $\mathfrak{p}$ 是素理想的矛盾。然而，对于形式化，我们发现通过对其范数施加越来越多的限制来简单地计算这两个理想的 GCD 会更方便。

`gcd_lemma` 的重要要素之一是，对于满足定理假设的 d ，Mordell 方程的任何解都必然有 x 为奇数。这个结论的最强相关版本实际上是：

```
lemma x_odd_two_three_aux {x y d : ℤ}
  (hd : ↑d ∈ ({2, 3, 5, 6, 7} : finset (ℤmod 8)))
  (h_eqn : y ^ 2 - d = x ^ 3) : ¬ even x
```

这可以通过推广到关于 $\mathbb{Zmod} 8$ 元素的陈述，并应用内置的决策过程 `dec_trivial` 来检查所有 $8 \times 8 \times 8$ 种情况来证明。请注意，这个引理对于 $d \equiv 5 \pmod{8}$ 也成立，因此这是当 $d \not\equiv 2, 3 \pmod{4}$ 时，这里形式化的方法可能适用的最相似的情况。

由此我们得到一个理想，它的立方是 $\langle y + \sqrt{d} \rangle$ ，并且根据关于类群的假设，它是一个主理想。从主理想的相等性过渡到它们在相差一个单位下的生成元的相等性，是在任何整环中都成立的交换代数中非常一般引理的应用。

对于 $d \leq -1$ 且 $d \equiv 2, 3 \pmod{4}$ 时 $\mathbb{Z}[\sqrt{d}]$ 的单位的计算，很快就可以从以下事实得出：范数

$$\text{Norm}(a + b\sqrt{d}) = a^2 - db^2$$

只有在 a, b, d 的值很小的情况下才能等于 1，这可以使用线性算术策略（例如 `linarith`）来处理。在所有情况下，我们都可以检查单位群的阶是二的幂，因此所有单位都是立方。

我们得出结论 $\exists z : \mathbb{Z}[\sqrt{d}], z^3 = \langle y, 1 \rangle$ 通过对 z 的分量 m, n 进行分情况讨论并应用 `quad_ring` 的外延性 (extensionality)，得到所需整数的等式：

$$(m^2 + 3 * d * n^2) * m = y$$

$$(d * n^2 + 3 * m^2) * n = 1$$

分解后发现 $n = \pm 1$ ，从而给出了定理陈述中存在量词的见证 (witness)。

这个定理的逆命题，即证明当条件 $d = -3m^2 \pm 1$ 对于某个 m 成立时，给定的 y 值实际上给出了一个解，这实际上在更普遍的情况下成立，并且原因要简单得多。所以我们这样陈述并证明它，只是简单地检查在任意交换环上这两种情况都给出了方程的解：

```
lemma Mordell_contra (d m : R)
  (hd : d = 1 - 3 * m^2 ∨ d = -1 - 3 * m^2) :
  (m * (d * 3 + m^2))^2 = (m^2 - d)^3 + d :=
by rcases hd with rfl | rfl; ring
```

现在我们来将 `Mordell_d` 应用于具体的情况。得益于第 4.3 节中对类数的计算，我们可以无条件地将上述结论应用于 $d \in \{-1, -2, -5, -6, -13\}$ 。这给出了以下定理：

```
theorem Mordell_minus1 (x y : ℤ)
  (h_eqn : y^2 = x^3 - 1) : y = 0
theorem Mordell_minus2 (x y : ℤ)
  (h_eqn : y^2 = x^3 - 2) : y = 5 ∨ y = -5
theorem Mordell_minus13 (x y : ℤ)
  (h_eqn : y^2 = x^3 - 13) : y = 70 ∨ y = -70
theorem Mordell_minus5 (x y : ℤ) : y^2 ≠ x^3 - 5
theorem Mordell_minus6 (x y : ℤ) : y^2 ≠ x^3 - 6
```

请注意，在前三种情况下，方程 $d = -3m^2 \pm 1$ 有解，分别对应 $m = 0$ ， $m = 1$ 和 $m = 2$ 。这些产生了所示的 y 值，这确实给出了 1 的解。在后两种情况下，不存在解。

在不存在解的结果以及其他没有解的 $d = -3m^2 \pm 1$ 情况中，我们都需要证明整数中的某些二次方程没有解。这归结为检查特定的有理数是否为平方数。虽然为这编写一个决策程序并不困难，但我们选择走捷径，简单地

检查所需的二次方程在某个精心选择的模 n 下没有解。这种方法的优点是, Lean 的 `mathlib` 已经表明环 `zmod n` 具有可判定的相等性并且是有限的, 因此内置的决策过程 `dec_trivial` 无需额外工作即可完成此类目标。这种方法的缺点是, 用户必须首先手动找到一个模 n , 使得所考虑的方程没有解。

5.2 初等和椭圆曲线方法

正如关于类群的计算一样, 讨论一些求解(某些)Mordell 方程的其他可能方法似乎是合适的。最重要的是, 这将使已经被形式化求解的 Mordell 方程处于某种透视中。

在不存在整数解的情况下, 有时可以使用较少的高级数论工具(例如二次互反律、整数的唯一分解和同余)来对无整数解给出更简单的证明[11, 第 XXIII 章]。这些证明更容易形式化, 因为技术前提条件较少, 但它们不能推广到包含更高级的有解情况。

方程 (1) 当作为 (x, y) 平面上的代数曲线看待时, 定义了一条椭圆曲线。Mordell 证明了任何椭圆曲线上具有整数坐标的点的数量(即定义方程的整数解)是有限的[23]。另一方面, 椭圆曲线上的有理点形成一个阿贝尔群, 称为曲线的 Mordell-Weil 群, 它是有限生成的, 通常具有无限阶的生成元。一般地确定椭圆曲线的秩是一个开放问题, 但目前在许多情况下可以有效地计算它。对于 Mordell 曲线, 挠何时出现是明确已知的。因此, 从一般角度来看, Mordell 方程最有趣的情况是那些曲线秩至少为一的情况, 以便存在无限多个有理解, 但只有有限多个整数解。

在下表中, 对于每个 $-1 \geq d \geq -13$ 的整数 d , 我们列出了与 d 对应的 Mordell 椭圆曲线 $E_d: y^2 = x^3 + d$ 相关的上述量, 即 E_d 的 Mordell-Weil 群的秩 $\text{rk}(E_d)$ 和其上的整数点数量 $|E_d(\mathbb{Z})|$, 以及二次域 $K_d = \mathbb{Q}(\sqrt{d})$ 的类数 $h(K_d)$ 。下表中的加粗元素表示已经形式化的结果。此外, 非无平方因子 d 的类数 $h(K_d)$ 的计算, 依赖于与由 d 的无平方因子部分的平方根生成的二次域的同构。

$-d$	1	2	3	4	5	6	7	8	9	10	11	12	13
$\text{rk}(E_d)$	0	1	0	1	0	0	1	0	0	0	2	0	1
$ E_d(\mathbb{Z}) $	1	2	0	4	0	0	4	1	0	0	4	0	2
$h(K_d)$	1	1	1	1	2	2	1	1	1	2	1	1	2

对于求解 (1), 当 $|E_d(\mathbb{Z})| \neq 0$ 时, 显然没有初等的非存在性证明可用, 当 $\text{rk}(E_d) \neq 0$ 时, 没有直接的椭圆曲线证明可用, 而在我们的方法中, 一旦 $h(K_d) \neq 1$, 就不可避免地需要类群。请注意, 情况 $d = -13$ 具备所有这三个特征。

6 计算策略

为了在 `quad_ring` 中实现计算, 我们使用了一对策略, 我们称之为 `quad_ring.calc_tac` 和 `times_table_tac`。我们最初使用的策略 `calc_tac` 是特定于 `quad_ring` 的, 需要更多手动指导。对于更复杂的目标, 我们实现了更强大、更通用的策略 `times_table_tac`。

6.1 `quad_ring.calc_tac`

利用我们的设置, 策略 `calc_tac` 可以通过现有的 `mathlib` 策略进行极其简短的实现:

```
meta def calc_tac : tactic unit :=
  `[\rw quad_ring.ext_iff;
    repeat { split; simp; ring }]
```

首先, 它使用外延性引理 (extensionality lemma) `quad_ring.ext_iff` 将一个方程 $x = y$ (其中 $x, y : \text{quad_ring } R \ a \ b$) 转化为连词 $x.b1 = y.b1 \wedge x.b2 = y.b2$ 。然后 `split` 策略将包含一个连词的单个目标替换为代表连词两侧的一对目标。`simp` 策略调用简化器来重写目标。最后, `ring` 策略(基于同名的 Coq 策略 [15])通过将方程两边重写为 Horner 范式来解决多项式表达式的相等性问题。因此, `calc_tac` 可以解决由 `quad_ring` 中的等式组成的任何目标, 只要简化器能够将目标转换为交换(半)环中多项式表达式的分量等式。

例如, 为了实例化 `comm_ring (quad_ring R a b)` 结构, 我们必须为字段 `mul_comm (x y : quad_ring R a b) : x * y = y * x` 提供一个值。

我们通过单次调用 `calc_tac` 来完成这一点。在应用外延性并拆分连词后, 该策略产生两个目标: $(x * y).b1 = (y * x).b1$ 和 $(x * y).b2 = (y * x).b2$ 。我们之前已经注册了指定乘积的各分量的 `@[simp]` 引理:

```
@[simp] lemma mul_b1 (z w : quad_ring F a b) :
  (z * w).b1 = z.b1 * w.b1 + z.b2 * w.b2 * b :=
  rfl
@[simp] lemma mul_b2 (z w : quad_ring F a b) :
  (z * w).b2 = z.b2 * w.b1 + z.b1 * w.b2 +
  z.b2 * w.b2 * a := rfl
```

当简化器遍历目标的子表达式时(从叶子节点开始), 它使用子表达式的头符号来查找所有相应的带有 `@[simp]` 标记的引理, 直到匹配成功。它执行重写并在新的子表达式中重新开始遍历, 直到整个表达式被详尽地重写。在这种情况下, 简化器将 `mul_b1` 和 `mul_b2` 各应用两次: 一次在两个目标的左侧, 一次在右侧。结果是两个目标: $x.b1 * y.b1 + x.b2 * y.b2 * b = y.b1 * x.b1 + y.b2 * x.b2 * b$ 以及 $x.b2 * y.b1 + x.b1 * y.b2 + x.b2 * y.b2 * a = y.b2 * x.b1 + y.b1 * x.b2 + y.b2 * x.b2 * a$ 。最后, 在两个目标上都调用了 `ring` 策略, 将两边视为变量 $x.b1 \ y.b1 \ x.b2 \ y.b2 \ a \ b$ 的多元多项式, 并利用环 R 的交换律将两边重写为相等的多项式, 从而完成证明。

显然, `calc_tac` 的适用性从根本上取决于简化器能否将 `quad_ring` 元素的分量约化为多项式表达式。换句话说, 为了能够应用 `calc_tac`, 用户必须策划一组足够强大的 `@[simp]` 引理, 以表达除多元多项式以外的所有计算。因此, 我们为 `quad_ring` 中的每个定义配对提供了一对引理, 分别指定其 `b1` 和 `b2` 分量; 这通常可以通过将 `@[sims]` 属性应用于定义来自动实现。另一方面, 由于简化器除了头符号之外不对集合进行任何索引, 因此它必须尝试所有引理直到找到适用的引理, 所以随着 `@[sims]` 引理集的增长, 它的速度明显变慢。这个顾虑

是一个相关因素，但并非决定性的，特别是由于 Lean 4 承诺使用区分树 (discrimination trees) 提供更高效的简化器 [9]。

`calc_tac` 结构的另一个缺点是在简化阶段可能会积累非常大的表达式。例如，`ring` 策略在目标 $(x - y) * (x^2 + x * y + y^2) = x^3 - y^3$ 上需要几十毫秒，而 `calc_tac` 需要几秒钟来证明 `quad_ring` 上同样的等式，因为它首先必须将每个乘法展开为乘法和，从而产生包含 72 次变量出现的中间结果。由于这些限制，我们只能将 `calc_tac` 应用于简单的方程，例如 `quad_ring` 的环公理。为了以更原则性的方式处理更复杂的等式，我们开发了一个更强大的策略原型，称为 `times_table_tac`。

6.2 乘法表 (Times tables)

`times_table_tac` 的目标是标准化交换环有限扩张中的表达式，包括但不限于 `quad_ring`。更准确地说，设 S/R 是交换环的一个扩张，使得 S 是自由且有限生成的 R -模，如果在 R 中给定了一个表达式标准化过程，那么 `times_table_tac` 可以标准化多元多项式环 $S[X_1, \dots, X_n]$ 中的表达式。

适用于 `times_table_tac` 的有限环扩张在数学中很常见，与本文特别相关的一个例子是 $S = R[\alpha]$ ，其中 α 是首一多项式的根，因为 `quad_ring` 是当 α 次数为 2 时 $R[\alpha]$ 的特例。满足 `times_table_tac` 所需前置条件的环扩张的其他常见例子包括：实数上的复数，给定环 R 上的所有矩阵环 $M_n(R)$ ，以及 $\mathbb{Z}/p\mathbb{Z}$ 上的所有有限域 $GF(p^k)$ 。

我们可以通过将每个 $x \in S$ 写成基向量族 b 的 R -线性组合来为 S 定义一种范式 (normal form)：如果 S 是一个自由的 R -模，那么这种线性组合的系数是良定义的并且是唯一的。这些线性组合的加法和标量乘法可以逐分量进行，但是对于 S 中两个元素的乘法，我们需要额外的信息。因为 R 的环扩张 S 也是一个 R -代数， S 中的乘法是一个 R -双线性映射，如果我们给定 S 作为 R -模的一个有限基 b （由于我们假设 S 是自由且有限的，因此存在），那么我们可以通过给出每对基元素 b_i, b_j 的乘积 $T_{i,j} \in S$ 来指定乘法操作。因此，为了将两个元素 $xy \in S$ 相乘，我们可以将每个元素写为基元素的线性组合 $x = \sum_i c_i b_i$ 和 $y = \sum_j d_j b_j$ ，由分配律，我们得到 $xy = \sum_{i,j} T_{i,j} c_i d_j$ 。记 $T_{i,j} = \sum_k t_{i,j,k} b_k$ ，则 xy 的范式由 $\sum_{i,j,k} c_i d_j t_{i,j,k} b_k$ 给出。基元素乘积的矩阵 T 是名称 `times_table_tac`（乘法表策略）的由来。

例如，如果我们设定 $S = \mathbb{Z}[\sqrt{d}]$ ，我们可以选择 $\{1, \sqrt{d}\}$ 作为基向量，乘法表由下式给出：

$$T = \begin{pmatrix} 1 & \sqrt{d} \\ \sqrt{d} & d \end{pmatrix}.$$

为了同时支持在表达式中使用变量，如果 S 是一个次数为 d 的 R -模，我们可以为 $S[X_1, \dots, X_n]$ 构造一种范式，通过在多项式环 $R[X_{1,0}, \dots, X_{1,d}, X_{2,0}, \dots, X_{n,d}]$ 中工作，并令 $X_i = \sum_j X_{i,j} b_j$ 。对于多元多项式环，一种合适的范式是 Horner 范式。

`times_table_tac` 的实现采用自底向上的方式为每个子表达式构造范式，当遇到它们时，将范式的和、标量

倍数和乘积求值为新的范式。在求值期间进行标准化可确保项保持相对较小，避免了 `calc_tac` 在更复杂的表达式上的问题。范式被记录为 Horner 范式的向量，重用 `ring` 策略的实现来处理 Horner 表达式 [15]。除了计算范式外，该策略还组合了一个证明，证明输入表达式等于其范式。在 Lean 3 中，构造证明项通常比通过反射证明 (proof by reflection) 更快，因为元程序的解释器通常比内核归约更有效。基的数据和关联的乘法表被捆绑在一个结构 `times_table` 中：

```
structure times_table (ι R S : Type*) [fintype ι]
  [semiring R] [add_comm_monoid S] [module R S]
  [has_mul S] :=
  (basis : basis ι R S)
  (table : ι → ι → ι → R)
  (unfold_mul : ∀ x y k, repr basis (x * y) k =
    ∑ i j : ι, repr basis x i * repr basis y j *
      table i j k)
```

字段 `basis` 和 `table` 分别对应于上面的变量 b 和 t 。这里，`repr basis` 是一个线性等价，它将向量 $x : S$ 映射到系数族 c ，使得 $\sum i, c_i \cdot \text{basis } i = x$ 。

我们以扩展 `norm_num` 策略的形式为 `times_table_tac` 提供了一个用户界面。此策略旨在通过将目标或假设中出现的数字表达式重写为显式数字来对其求值。具体而言，`times_table_tac` 扩展可评估形式为 `repr (times_table.basis T) x` 的表达式。因此，乘法表通过我们要标准化的表达式传递给策略。

请注意，`times_table` 的定义并不假设 S 具有环结构。这允许我们使用该策略来证明环公理，只需提供乘法表的显式乘法定义，就像我们使用 `calc_tac` 在 `quad_ring` 上构造 `comm_ring` 实例一样。

因为 `quad_ring.calc_tac` 被证明足以满足我们目前的目的，我们并没有在实践中应用 `times_table_tac`，而是将其作为一个概念验证留存。从我们的测试用例中，我们确定随着表达式变大，`calc_tac` 花费的时间明显长于 `times_table_tac`：在只有少数操作的目标上，例如 $\mathbb{Q}(\sqrt{d})$ 中乘法的交换律或结合律，`calc_tac` 大约比 `times_table_tac` 快两倍，而对于诸如 $x^4 - y^4 = (x - y)(x^3 + x^2y + xy^2 + y^3)$ 这样较大的目标，该比率降至 $\frac{3}{2}$ 。不过，这意味着两种策略随着表达式的增长都会出现明显的减速：它们需要一分多钟才能解决这个相对简单的目标。不幸的是，约简诸如 $\sum_{i,j,k} c_i d_j t_{i,j,k} b_k$ 之类求和的标准化步骤占用了很多时间，以至于在我们开发过程中遇到的小表达式上 `calc_tac` 反而更快。我们相信，在实现 `times_table_tac` 的完整化身时，良好的缓存策略可以解决这些问题，因为相同的表达式 c_i, d_j 和 $t_{i,j,k}$ 在展开定义时往往会重复出现。

7 连接到计算机代数系统

如上所述，Mordell 方程的解和类群的计算涉及数域和环中的许多计算。例如，为了验证两个理想在理想类群中是否互为逆，我们必须展开其生成元的几个乘积，并通常检查其中一个是否整除所有其他生成元。

许多此类工作耗费在常规计算上,可以采用算法完成。特别地,许多此类计算一旦执行,其认证过程可能比执行过程更快。例如,检查元素是否属于理想,或更一般地,检查由显式生成元列表给出的两个理想之间的包含关系或相等性,可能是一项不简单的任务,这取决于基环,但只要可以在环中进行有效的算术运算,就可以始终对其进行认证。这向形式化者提出了一个选择:要么在使用的证明助手或其元语言中直接实现此类算法的计算效率高的版本,要么在现有的计算机代数系统中进行这些计算,然后以某种方式导入并验证结果。

直接实现这些计算可能极其困难且耗时。根据所需的计算,在证明助手中实现最先进的算法可能需要好几个月的时间。从外部工具导入计算可以在一定程度上通过手动完成,但这会增加大量开销,而且如果涉及的项很大,还会降低可读性。

为了自动执行在证明过程中调用外部系统的过程,我们开发了一个策略,该策略使用 SageMath 计算机代数系统 [31] 作为后端,来决定插入具有更有用形式的替换数据,这些数据要么以某种方式被简化,要么被分解为更小的部分,从而使项的某些属性更加清晰。主要策略 `replace_certified_sage_equality` 将在下面描述。它旨在成为一种灵活的方式,允许用户添加自己的策略,使用 SageMath 进行未经验证的计算,然后仅需几行代码即可验证它们。该系统在目标上类似于 [18, 20],但我们旨在使我们的系统在某些方面得到更广泛的使用。

该系统被设计为灵活的,因此它可以用于实现许多符合此配置文件的策略。例如,我们可以实现因子分解自然数和多项式、分解为可除部分和余数,或者将矩阵替换为基变换矩阵乘上某种范式矩阵的策略。有关如何实现对 SageMath 的访问的详细信息,请参见 section A。我们打算在工作中需要明确计算的部分使用这种范式,但目前尚未这样做。

设计 `replace_certified_sage_equality` 的签名如下:

```
meta def replace_certified_sage_equality
  (matcher : expr → tactic bool)
  (reify_type : Type*)
  (reify : expr → tactic reify_type)
  (sage_input : reify_type → tactic string)
  (output_type : Type*)
  [has_from_sage output_type]
  (convert_output : output_type → tactic expr)
  (validator : expr → expr → tactic (expr))
  (name : string)
  (wi : parse (tk "with" *> pexpr)?):
  tactic unit
```

这个 `replace_certified_sage_equality` 策略在目标上迭代,并寻找 `matcher` 返回 `true` 的子表达式。找到的第一个子表达式然后使用 `reify` 被具体化 (`reified`) 为 `reify_type` 类型的显式数据;这意味着即使是类型不可计算的项,只要 `reify` 函数能够识别该项,也可以被具体化。然后 `sage_input` 函数被用于创建一个字符串,即传递给 SageMath 的命令。SageMath 的输出(通过其中一个后端)被读取为一个字符串,然后被解析为 `output_type`

(这种解析通过类型类处理,以方便解析诸如嵌套元组和列表之类的类型)。然后 `convert_output` 负责构造一个表达式,该表达式表示与最初匹配的输入类型相同的项,这两者的相等性随后由 `validator` 证明,该验证器返回相等性的证明。然后通过重写以将最初匹配的表达式替换为 SageMath 返回的经过验证的版本。最后两个参数用于解析策略之后的可选 `with "a string"`,它用于自替换 (`self replaced`) 版本中。

使用这个样板策略,一个使用 SageMath 对自然数进行分解的策略仅用约 20 行即可实现(参见 section B),而实现者无需知道调用 SageMath 的细节。对于此示例,我们将表示自然数的表达式具体化为自然数本身,并要求 SageMath 给出作为素因数和指数对列表的因子分解。然后,我们将相应的分解项构造为幂的乘积。`mathlib` 策略 `norm_num` 用于证明原始表达式与结果表达式的相等性。

8 讨论

8.1 未来工作

除了第 4.4 节中提到的方法(特别是使用 Minkowski 界)之外,还有几种自然的方法可以扩展目前的工作。

可以进行修改以考虑其他 Mordell 方程,包括以下内容。当 $d = -7$ 时,这将涉及考虑模 4 余 1 的判别式,并因此在 $\text{quad_ring } \mathbb{Z} [1, (d - 1) / 4]$ 上工作。这应该不会比上面的证明复杂得多。当 $d = -23$ 时,这将涉及处理 $h(K) = 3$ 。为此,我们需要对类群的元素使用显式的理想代表(原则上我们需要进行计算)。当 $d > 0$ 时,这将涉及更多的工作,因为 $\mathbb{Q}(\sqrt{d})$ 的整数环的单位群(假设 d 不是平方数)此时是无限的。模掉立方后单位的可能性仍然是有限的,但是除了计算单位群的生成元的问题之外,从主理想过渡到元素时,这将需要一些额外的分情况讨论。

我们也可以考虑更一般形式的方程。上面讨论的技术对于求解比 Mordell 方程更一般的方程仍然有用。例如, $y^2 = x^3 + d$ 中的指数 2, 3 可以改变,或者更普遍地,可以考虑形式为 $y^d = f(x)$ 的超椭圆方程,其中 $f \in \mathbb{Q}[x]$ 为任意多项式。这涉及在代数 $\mathbb{Q}[x]/\langle f \rangle$ 中工作,并且至少需要在计算类群和单位群的已验证算法方面取得重大进展。

在策略方面,很明显在显式数域和理想中工作时产生的许多目标将具有相当特定的形式,例如检查数域中的某些等式或计算、检查两个理想的相等性或寻找其规范化的生成元集、检查类群中理想的等价性或证明给定理想的非主性。解决这些目标的专门策略应该指日可待,并将极大加快形式化该领域论证的进度。

求解 Mordell 方程的一个应用是能够确定所有在 $\mathbb{Q}$ 上具有规定不良还原 (`bad reduction`) 的椭圆曲线。这也可以在未来的工作中形式化。

8.2 相关工作

以前关于丢番图方程的工作主要包括初等解法,例如 Fermat、Euler、Gauss 和 Legendre 的工作。这类材料有几种形式化,涉及显式变量代换、检查模素数的解、二次

互反律以及在 $\mathbb{Z}$ 内的下降。例如，费马最后定理的指数 4 情况已经在几个证明助手中被形式化，包括 Isabelle/HOL（见 [26]，也适用于指数 3 的情况）、Coq（见 [10]）和 Lean³。

已经有几个关于从数理逻辑角度来看待丢番图方程和问题的形式化工作（例如，参见 [17] 或 [27] 及其参考文献）。该领域与这里研究的基本对象和问题有一些重叠，但在方法和目标方面仍然截然不同。

Pell 方程 ($x^2 - dy^2 = 1$) 的解是 Freek Wiedijk 跟踪的形式化目标列表中列出的“前 100 个”数学定理之一，该方程的某些方面已在 HOL Light、Isabelle/HOL、Metamath、Lean 和 Mizar 中形式化。这与计算实二次域中的单位群有关，单位群在求解正数 d 的 Mordell 方程时发挥作用。然而，Lean 的 mathlib 中的形式化专门针对 $d = a^2 - 1$ 对于某个 $a \in \mathbb{Z}$ 的情况。这使得它只对求解 Mordell 方程时出现的一小部分实二次域有用，因此如果不进行推广，作为丢番图应用的形式化，它并没有发挥出应有的作用。

也有一些致力于与费马最后定理相关形式化的倡议，最著名的是正在进行的“FLT regular”项目⁴，旨在形式化 Kummer 在 19 世纪对所谓正则素数指数的费马最后定理的工作。

De Frutos-Fernández 形式化了数域的 adèles 和 idèles 理论 [7]，它提供了另一种关于理想类群的更理论化的思考方式，并开启了类域论的发展。

8.3 结论

我们成功形式化了几个 Mordell 方程的解，更重要的是，建立了可用于求解其他 Mordell 方程实例或一般丢番图方程的理论和策略。我们发现，我们的大部分精力都花在了计算的每个细节上，例如通常留给计算机代数系统的理想的相等性计算，我们设计了一些策略的原型以使将来更容易做到这一点。另一方面，更理论化的论证，例如从 $\sqrt{2}$ 范数的素数性推导出该理想的素数性，形式化起来非常顺利。

总的来说，我们的开发占用了大约 6000 行代码，其中大约 1000 行涉及预备知识，其余大致平均分布在本文的第 3—6 节。像这样一个项目的 De Bruijn 因子很难作为一个整体进行可靠地测量。这是因为这里形式化的结果在论文和形式化系统中都有许多先决条件，没有一本书或一篇文章是从几乎没有数学背景开始并呈现这里形式化的结果。因此，对 De Bruijn 因子的任何测量都很大程度上取决于作者选择将什么视为背景材料或非正式工作中的隐含内容。

Acknowledgments

Anne Baanen 得到了 NWO Vidi 资助编号 016.Vidi.189.037，Lean Forward 的资助。Alex J. Best, Nirvana Coppola 和 Sander R. Dahmen 得到了 NWO Vidi 资助编号 639.032.613，New Diophantine Directions 的资助。

³https://github.com/leanprover-community/mathlib/blob/4e1eebe/src/number_theory/fermat4.lean

⁴<https://github.com/leanprover-community/flt-regular>

我们要感谢 Jasmin Blanchette 对论文早期版本的评论。感谢匿名审稿人的有用评论。最后，我们要感谢 mathlib 社区在整个开发过程中的支持。

A 附录：访问系统

我们相信使用免费和开源的计算机代数系统作为后端有助于确保该策略可能被许多用户使用。为了增加此类策略在特定系统上无需额外设置即可使用的可能性，我们实现了几个后端，以直接或通过 conda⁵ 安装的方式在用户机器上调用 SageMath。如果用户机器上没有安装 SageMath，我们添加了三个后端以通过 curl、ncat 或 openssl 访问互联网上的公共 SageMath 实例，网址为 <https://sagecell.sagemath.org>。后一种方法以前曾被 Dhruv Bhatia 编写的 mathlib 策略 polyrith 所使用。这些是我们选择使用 SageMath 来实现这些策略的主要原因，尽管由于界面是纯文本，将其替换为其他系统也应该很简单，无需进行大的更改。

除了确保可以使用多种方式访问 SageMath 之外，我们还以自替换 (self-replacing) 的方式设计了该策略。由于 SageMath 返回字符串输出，因此一旦运行该策略一次，将来对它的使用就可以直接提供适当的 SageMath 输出。

例如，使用通过此架构实现的分解自然数策略，建议用户将对策略的调用替换为包含 SageMath 显式输出的调用：

```
example : ¬ nat.prime 1111 := begin
  factor_nats, -- 尝试这个: factor_nats with
    "[ (11, 1), (101, 1) ]"
  simp [nat.prime_mul_iff], -- a product can only
    be prime if all factors are one
end
```

B 附录：用于分解因式的 CAS 认证策略示例

```
meta def tactic.interactive.factor_nats :=
  replace_certified_sage_equality
  (λ ex, do
    t ← infer_type ex,
    return $ t = `(nat))
N
(λ e, e.to_nat)
(λ n, return
  sformat!"print(list(ZZ({n}).factor()))")
(list (N × N))
(λ l, do
  ini ← l.mfoldl (λ ol ⟨p, n⟩, do
    P ← expr.of_nat `(N) p,
    N ← expr.of_nat `(N) n,
    return `((%ol : N) * (%P : N) ^ (%N :
      N))) `(1 : N),
  sl ← simp_lemmas.mk_default,
  prod.fst <$> simplify sl [] ini)
(λ o n, do
```

⁵<https://docs.conda.io>

```

(e1', p1) ← or_refl_conv norm_num.derive o,
(e2', p2) ← or_refl_conv norm_num.derive n,
is_def_eq e1' e2',
mk_eq_symm p2 >= mk_eq_trans p1)
"factor_nats"

```

References

- [1] Reynald Affeldt, Cyril Cohen, Marie Kerjean, Assia Mahboubi, Damien Rouhling, and Kazuhiko Sakaguchi. 2020. Competing Inheritance Paths in Dependent Type Theory: A Case Study in Functional Analysis. In *IJCAR 2020 (LNCS, Vol. 12167)*, Nicolas Peltier and Viorica Sofronie-Stokkermans (Eds.). Springer, 3–20. https://doi.org/10.1007/978-3-030-51054-1_1
- [2] Anne Baanen. 2022. Use and Abuse of Instance Parameters in the Lean Mathematical Library. In *ITP 2022 (LIPIcs, Vol. 237)*, June Andronick and Leonardo de Moura (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 4:1–4:20. <https://doi.org/10.4230/LIPIcs.ITP.2022.4>
- [3] Anne Baanen, Sander R. Dahmen, Ashvni Narayanan, and Filippo A. E. Nuccio Mortarino Majno di Capriglio. 2021. A Formalization of Dedekind Domains and Class Groups of Global Fields. In *ITP 2021 (LIPIcs, Vol. 193)*, Liron Cohen and Cezary Kaliszyk (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 5:1–5:19. <https://doi.org/10.4230/LIPIcs.ITP.2021.5>
- [4] Alan Baker. 1968. Contributions to the Theory of Diophantine Equations. II. the Diophantine Equation $y^2 = x^3 + k$. *Philosophical Transactions of the Royal Society of London. Series A, Mathematical and Physical Sciences* 263, 1139 (1968), 193–208. <http://www.jstor.org/stable/73429>
- [5] Michael A. Bennett and Amir Ghadermarzi. 2015. Mordell’s equation: a classical approach. *LMS Journal of Computation and Mathematics* 18, 1 (2015), 633–646. <https://doi.org/10.1112/S1461157015000182>
- [6] Henri Cohen. 2007. *Number theory: Volume I: Tools and Diophantine equations*. Graduate Texts in Mathematics, Vol. 239. Springer, Cham. <https://doi.org/10.1007/978-0-387-49923-9>
- [7] María Inés de Frutos-Fernández. 2022. Formalizing the Ring of Adèles of a Global Field. In *ITP 2022 (LIPIcs, Vol. 237)*, June Andronick and Leonardo de Moura (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 14:1–14:18. <https://doi.org/10.4230/LIPIcs.ITP.2022.14>
- [8] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean theorem prover (system description). In *Automated Deduction - CADE-25 (LNCS, Vol. 9195)*, A. P. Felty and A. Middeldorp (Eds.). Springer, 378–388. https://doi.org/10.1007/978-3-319-21401-6_26
- [9] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction - CADE-28 (LNCS, Vol. 12699)*, André Platzer and Geoff Sutcliffe (Eds.). Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37
- [10] David Delahaye and Micaela Mayero. 2005. Diophantus’ 20th Problem and Fermat’s Last Theorem for $n = 4$: Formalization of Fermat’s Proofs in the Coq Proof Assistant. Draft. <http://arxiv.org/abs/cs/0510011>.
- [11] Leonard E. Dickson. 1920. *History of the theory of numbers*. Vol. 2. Carnegie Institution of Washington.
- [12] David S. Dummit and Richard M. Foote. 2004. *Abstract algebra* (third ed.). John Wiley & Sons, Inc., Hoboken, NJ. xii+932 pages.
- [13] Manuel Eberl. 2017. Minkowski’s Theorem. *Archive of Formal Proofs* (July 2017). https://isa-afp.org/entries/Minkowskis_Theorem.html, Formal proof development.
- [14] Harold M. Edwards. 2000. *Fermat’s last theorem*. Graduate Texts in Mathematics, Vol. 50. Springer, Cham. xvi+410 pages. <https://doi.org/10.1007/978-0-387-95002-0>
- [15] Benjamin Grégoire and Assia Mahboubi. 2005. Proving equalities in a commutative ring done right in Coq. In *TPHOLS 2005 (LNCS, Vol. 3603)*, J. Hurd and T. Melham (Eds.). Springer, Cham, 98–113. https://doi.org/10.1007/11541868_7
- [16] S. Lang. 2005. *Undergraduate Algebra*. Springer New York.
- [17] Dominique Larchey-Wendling and Yannick Forster. 2019. Hilbert’s Tenth Problem in Coq. In *FSCD 2019 (LIPIcs, Vol. 131)*, Herman Geuvers (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:20. <https://doi.org/10.4230/LIPIcs.FSCD.2019.27>
- [18] Robert Y. Lewis. 2017. An Extensible Ad Hoc Interface between Lean and Mathematica. *Electronic Proceedings in Theoretical Computer Science* 262 (Dec. 2017), 23–37. <https://doi.org/10.4204/EPTCS.262.4> arXiv:1712.09288 [cs].
- [19] Robert Y. Lewis and Paul-Nicolas Madelaine. 2020. Simplifying Casts and Coercions (Extended Abstract). In *Practical Aspects of Automated Reasoning (CEUR Workshop Proceedings, Vol. 2752)*, Pascal Fontaine, Konstantin Korovin, Ilias S. Kotsireas, Philipp Rümmer, and Sophie Tourret (Eds.). CEUR-WS.org, Aachen, Germany, 53–62. <http://ceur-ws.org/Vol-2752/paper4.pdf>
- [20] Robert Y. Lewis and Minchao Wu. 2022. A Bi-Directional Extensible Interface Between Lean and Mathematica. *Journal of Automated Reasoning* 66, 2 (May 2022), 215–238. <https://doi.org/10.1007/s10817-021-09611-1>
- [21] Louis Mordell. 1914. The Diophantine Equation $y^2 - k = x^3$. *Proceedings of the London Mathematical Society* s2-13, 1 (01 1914), 60–80. <https://doi.org/10.1112/plms/s2-13.1.60>
- [22] Louis Mordell. 1920. A Statement by Fermat. *Proceedings of the London Mathematical Society* s2-18 (1920), v–vi. <https://doi.org/10.1112/plms/s2-18.1.1-s>
- [23] Louis Mordell. 1923. On the Integer Solutions of the Equation $ey^2 = ax^3 + bx^2 + cx + d$. *Proceedings of the London Mathematical Society* s2-21, 1 (01 1923), 415–419. <https://doi.org/10.1112/plms/s2-21.1.415>
- [24] Louis Mordell. 1969. *Diophantine equations*. Academic Press.
- [25] J. Neukirch. 1999. *Algebraic number theory*. Fundamental Principles of Mathematical Sciences, Vol. 322. Springer, Cham. xviii+571 pages. <https://doi.org/10.1007/978-3-662-03983-0> Translated from the 1992 German original and with a note by Norbert Schappacher, With a foreword by G. Harder.
- [26] R. Oosterhuis. 2007. Mechanised theorem proving: exponents 3 and 4 of Fermat’s last theorem using Isabelle. MSc thesis. Faculty of Science and Engineering, Groningen. <https://fse.studenttheses.ub.rug.nl/8392/>
- [27] Karol Pāk and Cezary Kaliszyk. 2022. Formalizing a Diophantine Representation of the Set of Prime Numbers. In *ITP 2022 (LIPIcs, Vol. 237)*, June Andronick and Leonardo de Moura (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:8. <https://doi.org/10.4230/LIPIcs.ITP.2022.26>
- [28] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. 2020. Tabled Typeclass Resolution. *CoRR* abs/2001.04301 (2020). arXiv:2001.04301 <https://arxiv.org/abs/2001.04301>
- [29] Harold Stark. 1973. Effective estimates of solutions of some Diophantine equations. *Acta Arithmetica* 24, 3 (1973), 251–259. <http://eudml.org/doc/205226>
- [30] The mathlib Community. 2020. The Lean Mathematical Library. In *CPP 2020 (Certified Programs and Proofs)*, J. Blanchette and C. Hrițcu (Eds.). ACM, 367–381. <https://doi.org/10.1145/3372885.3373824>
- [31] The Sage Developers. 2022. *SageMath, the Sage Mathematics Software System (Version 9.7)*. <https://www.sagemath.org>.